Introduktion til AI

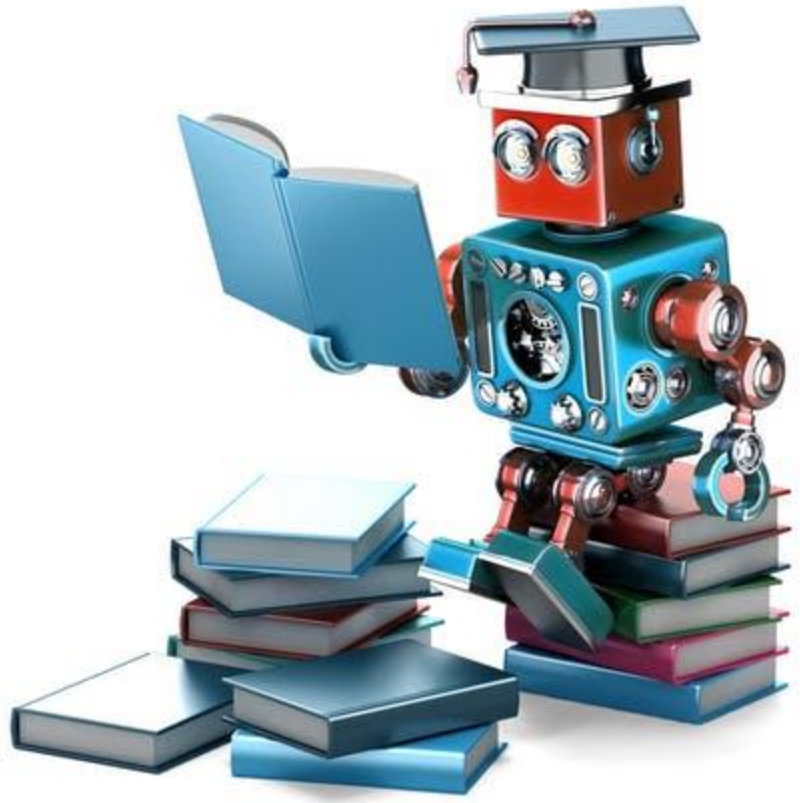


Professor Lasse Riis Østergaard, AAU
Dept. of Health Science and Technology
Medical Informatics and Image Analysis

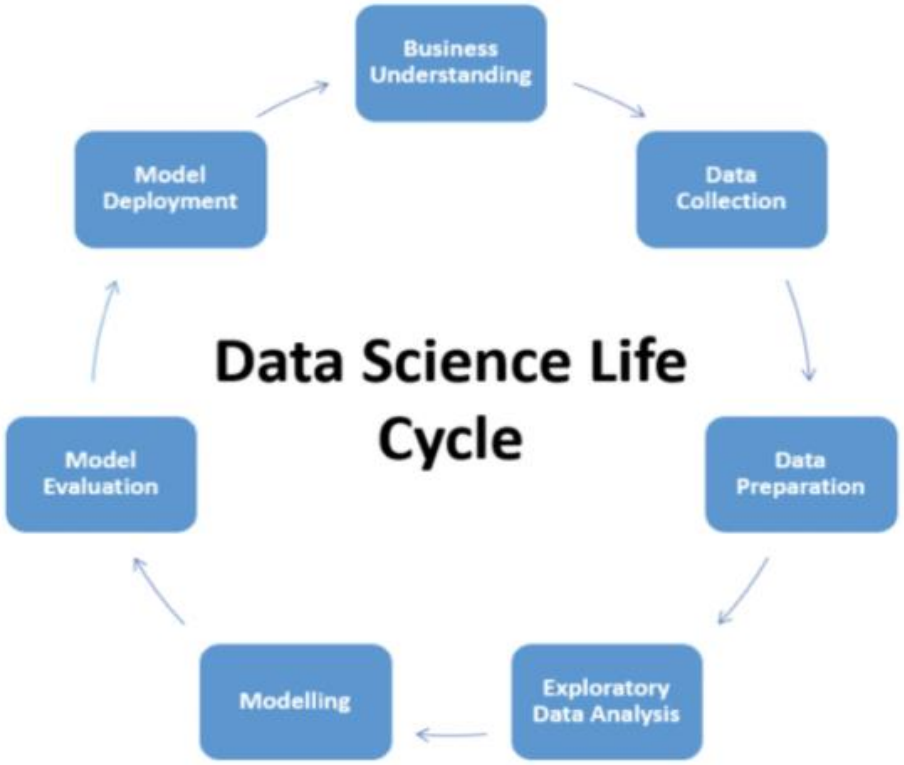
AALBORG UNIVERSITY
DENMARK

Agenda

- End-to-End Machine Learning (part 2)
  - Prepare data
  - Select and train model
  - Fine-tune model
  - Launch, monitor and maintain system
- Opgaver



The data science life cycle



1. Frame the problem and look at the big picture.
2. Get the data.
3. Explore the data to gain insights.
4. Prepare the data to better expose the underlying data patterns to machine learning algorithms.
5. Explore many different models and shortlist the best ones.
6. Fine-tune your models and combine them into a great solution.
7. Present your solution.
8. Launch, monitor, and maintain your system.

Prepare the Data - checklist

- Work on copies of the data (keep the original dataset intact).
- Write functions for all data transformations you apply, for five reasons:
  - So you can easily prepare the data the next time you get a fresh dataset
  - So you can apply these transformations in future projects
  - To clean and prepare the test set
  - To clean and prepare new data instances once your solution is live
  - To make it easy to treat your preparation choices as hyperparameters

1. Clean the data:
   - Fix or remove outliers (optional).
   - Fill in missing values (e.g., with zero, mean, median…) or drop their rows (or columns).
2. Perform feature selection (optional):
   - Drop the attributes that provide no useful information for the task.
3. Perform feature engineering, where appropriate:
   - Discretize continuous features.
   - Decompose features (e.g., categorical, date/time, etc.).
   - Add promising transformations of features (e.g., log(x), sqrt(x), x2, etc.).
   - Aggregate features into promising new features.
4. Perform feature scaling:
   - Standardize or normalize features.

Separate the predictors and the labels

- you don't necessarily want to apply the same transformations to the predictors and the target values

```
housing = strat_train_set.drop("median_house_value", axis=1)
housing_labels = strat_train_set["median_house_value"].copy()
```

Clean the Data

- total_bedrooms attribute has some missing values

1. Get rid of the corresponding districts.

```
housing.dropna(subset=["total_bedrooms"], inplace=True)
```

2. Get rid of the whole attribute.

```
housing.drop("total_bedrooms", axis=1)
```

3. Set the missing values to some value (zero, the mean, the median, etc.). This is called imputation.

```
median = housing["total_bedrooms"].median()  # option 3
housing["total_bedrooms"].fillna(median, inplace=True)
```

```
>>> housing.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 10 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   longitude           20640 non-null  float64
 1   latitude            20640 non-null  float64
 2   housing_median_age  20640 non-null  float64
 3   total_rooms         20640 non-null  float64
 4   total_bedrooms      20433 non-null  float64
 5   population          20640 non-null  float64
 6   households          20640 non-null  float64
 7   median_income       20640 non-null  float64
 8   median_house_value  20640 non-null  float64
 9   ocean_proximity     20640 non-null  object
dtypes: float64(9), object(1)
memory usage: 1.6+ MB
```

Imputation

- Many real world datasets contain missing values, often encoded as blanks, NaNs or other placeholders
- Imputed with
  - Mean
  - Median
  - Most frequent (non-numerical data)
  - Constant value (non-numerical data)
- Impute values on
  - Training set
  - Validation set
  - Test set
  - New data

	Car	MPG	Cylinders	Displacement	Horsepower	Weight	Acceleration	Model	Origin
0	STRING	DOUBLE	INT	DOUBLE	DOUBLE	DOUBLE	DOUBLE	INT	CAT
1	Chevrolet Chevelle Malibu	NaN	8	307.0	130.0	3504.	12.0	70	US
2	Buick Skylark 320	15.0	8	350.0	NaN	3693.	11.5	70	US
3	Plymouth Satellite	NaN	8	318.0	150.0	3436.	11.0	70	US
4	AMC Rebel SST	16.0	8	NaN	150.0	NaN	12.0	70	US
5	Ford Torino	17.0	8	302.0	140.0	3449.	10.5	70	US
6	Ford Galaxie 500	NaN	8	429.0	NaN	4341.	10.0	70	US
7	Chevrolet Impala	14.0	8	454.0	220.0	4354.	9.0	70	US
8	Plymouth Fury iii	14.0	8	440.0	215.0	4312.	8.5	70	US
9	Pontiac Catalina	14.0	8	455.0	225.0	4425.	10.0	70	US

```
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy="median")
```

Handling Text and Categorical Attributes

```
>>> housing_cat = housing[["ocean_proximity"]]
>>> housing_cat.head(8)
       ocean_proximity
13096         NEAR BAY
14973        <1H OCEAN
3785            INLAND
14689           INLAND
20507       NEAR OCEAN
1286            INLAND
18078        <1H OCEAN
4396          NEAR BAY
```

```
from sklearn.preprocessing import OrdinalEncoder

ordinal_encoder = OrdinalEncoder()
housing_cat_encoded = ordinal_encoder.fit_transform(housing_cat)
```

```
>>> housing_cat_encoded[:8]
array([[3.],
       [0.],
       [1.],
       [1.],
       [4.],
       [1.],
       [0.],
       [3.]])
```

Label Encoding

Food Name	Categorical #	Calories
Apple	1	95
Chicken	2	231
Broccoli	3	50

→

One Hot Encoding

Apple	Chicken	Broccoli	Calories
1	0	0	95
0	1	0	231
0	0	1	50

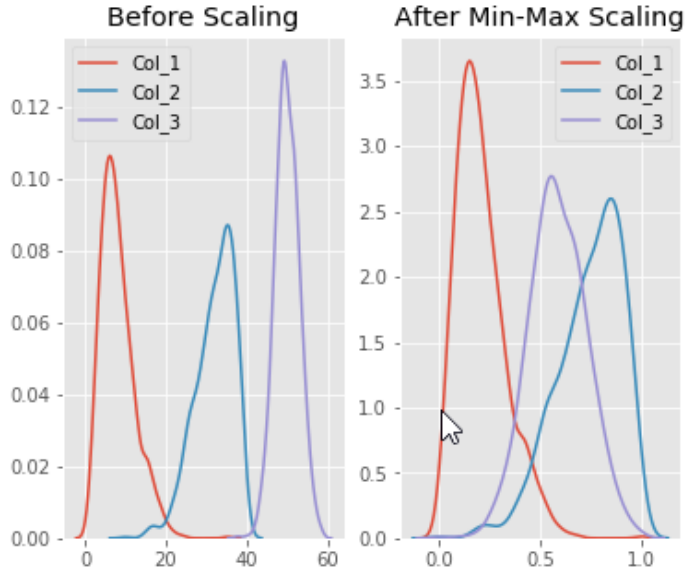
```
from sklearn.preprocessing import OneHotEncoder

cat_encoder = OneHotEncoder()
housing_cat_1hot = cat_encoder.fit_transform(housing_cat)
```

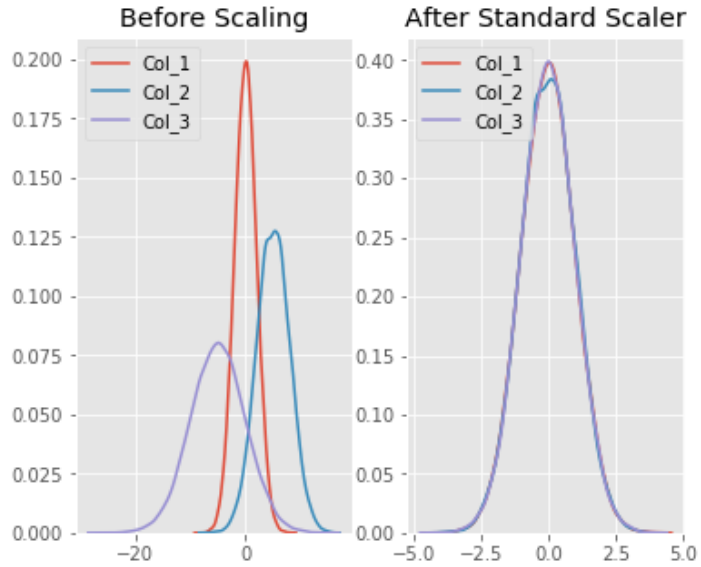
```
>>> housing_cat_1hot.toarray()
array([[0., 0., 0., 1., 0.],
       [1., 0., 0., 0., 0.],
       [0., 1., 0., 0., 0.],
       ...,
       [0., 0., 0., 0., 1.],
       [1., 0., 0., 0., 0.],
       [0., 0., 0., 0., 1.]])
```

Feature Scaling and Transformation

- Feature scaling is one of the most important transformations
- Machine learning algorithms don't perform well when the input numerical attributes have very different scales
- Two common ways to get all attributes to have the same scale: min-max scaling and standardization

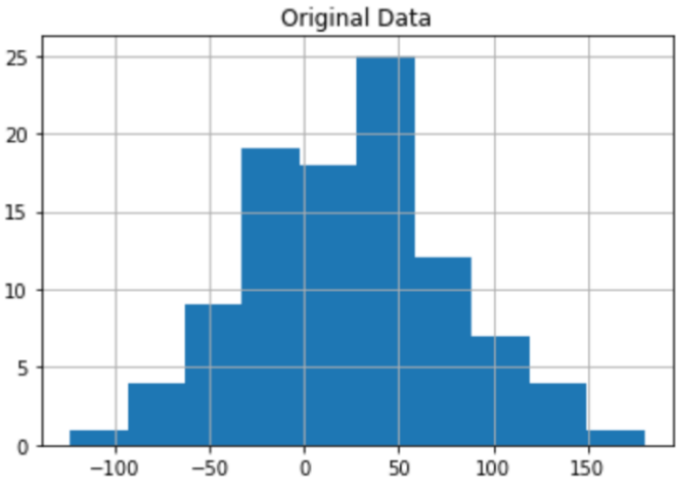


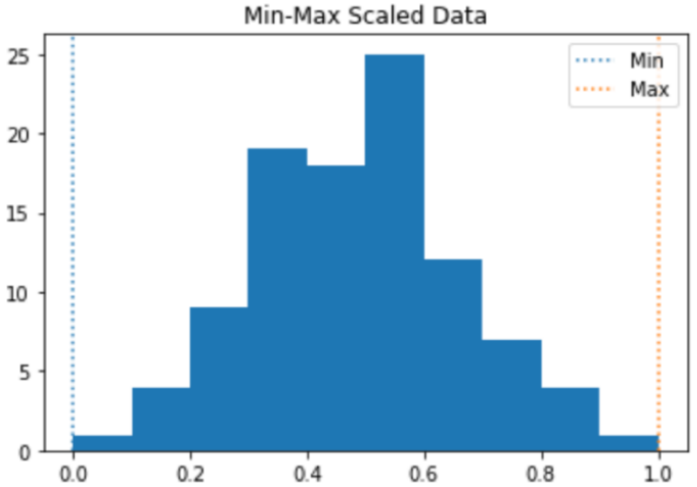
$$\frac{x_i - min(x)}{max(x) - min(x)}$$

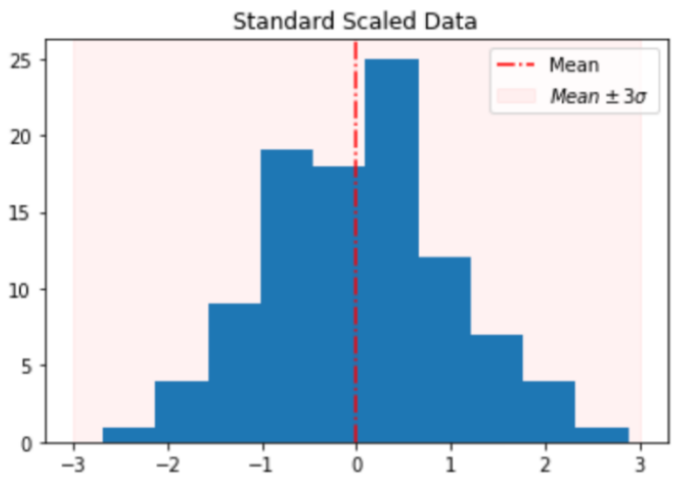


$$\frac{x_i - mean(x)}{stdev(x)}$$

Scaling


Original Data
25
20
15
10
5
0
−100
−50
0
50
100
150


Min-Max Scaled Data
Min
Max
25
20
15
10
5
0
0.0
0.2
0.4
0.6
0.8
1.0


Standard Scaled Data
Mean
Mean ± 3σ
25
20
15
10
5
0
−3
−2
−1
0
1
2
3

Transformation

- Heavy tail distributions
- Transform to make the distribution symmetrical

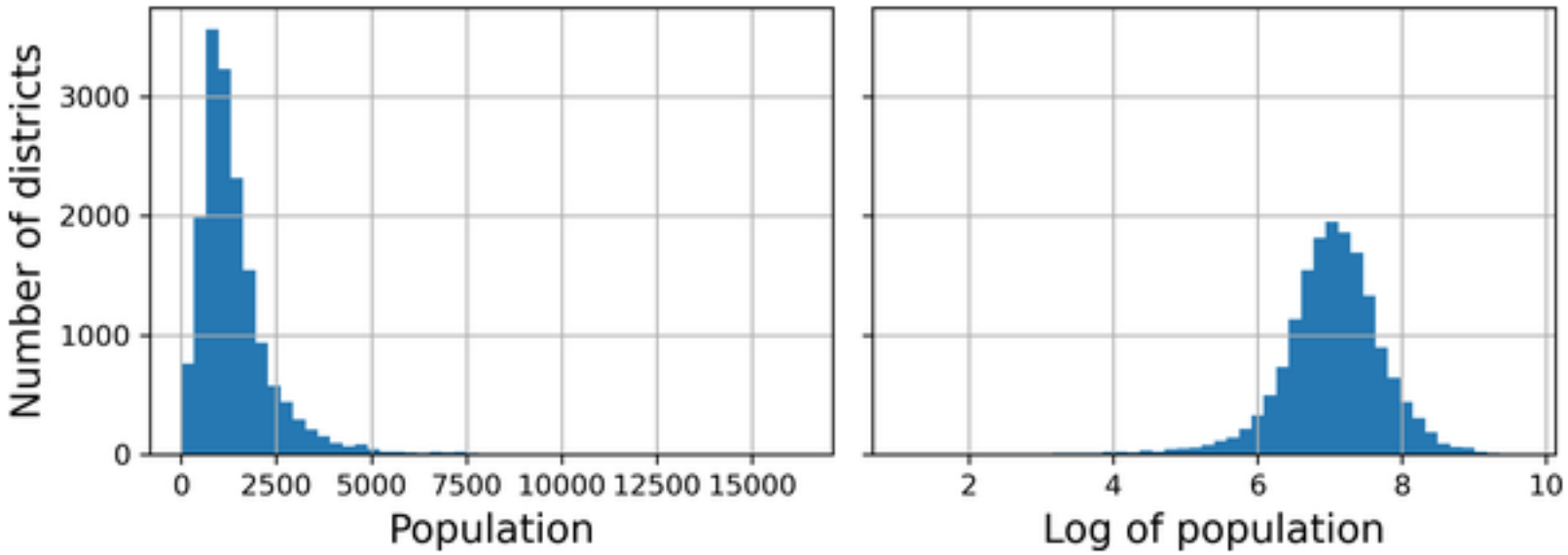


Figure 2-17. Transforming a feature to make it closer to a Gaussian distribution

Shortlist Promising Models - checklist

- If the data is huge, you may want to sample smaller training sets so you can train many different models in a reasonable time (be aware that this penalizes complex models such as large neural nets or random forests).
- Once again, try to automate these steps as much as possible.

1. Train many quick-and-dirty models from different categories (e.g., linear, naive Bayes, SVM, random forest, neural net, etc.) using standard parameters.
2. Measure and compare their performance:
   - For each model, use N-fold cross-validation and compute the mean and standard deviation of the performance measure on the N folds.
3. Analyze the most significant variables for each algorithm.
4. Analyze the types of errors the models make:
   - What data would a human have used to avoid these errors?
5. Perform a quick round of feature selection and engineering.
6. Perform one or two more quick iterations of the five previous steps.
7. Shortlist the top three to five most promising models, preferring models that make different types of errors.

Select and Train a Model - Train and Evaluate on the Training Set

- First five predictions

```
>>> housing_predictions = lin_reg.predict(housing)
>>> housing_predictions[:5].round(-2)  # -2 = rounded to the nearest hundred
array([243700., 372400., 128800.,  94400., 328300.])
>>> housing_labels.iloc[:5].values
array([458300., 483800., 101700.,  96100., 361800.])
```

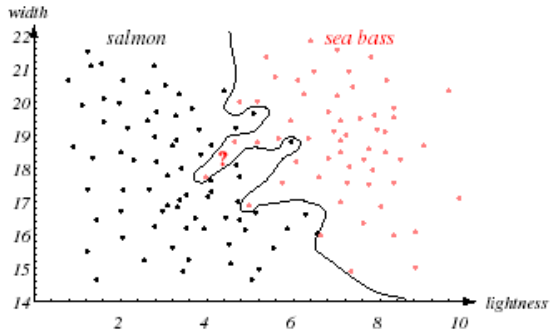
- RMSE as performance measure
  - Measures the average magnitude of the errors between predicted values and actual values in a dataset (Euclidian)

```
>>> from sklearn.metrics import mean_squared_error
>>> lin_rmse = mean_squared_error(housing_labels, housing_predictions,
...                               squared=False)
...
>>> lin_rmse
68687.89176589991
```

- Decision tree regression

```
>>> housing_predictions = tree_reg.predict(housing)
>>> tree_rmse = mean_squared_error(housing_labels, housing_predictions,
...                                squared=False)
...
>>> tree_rmse
0.0
```

- Overfitting?

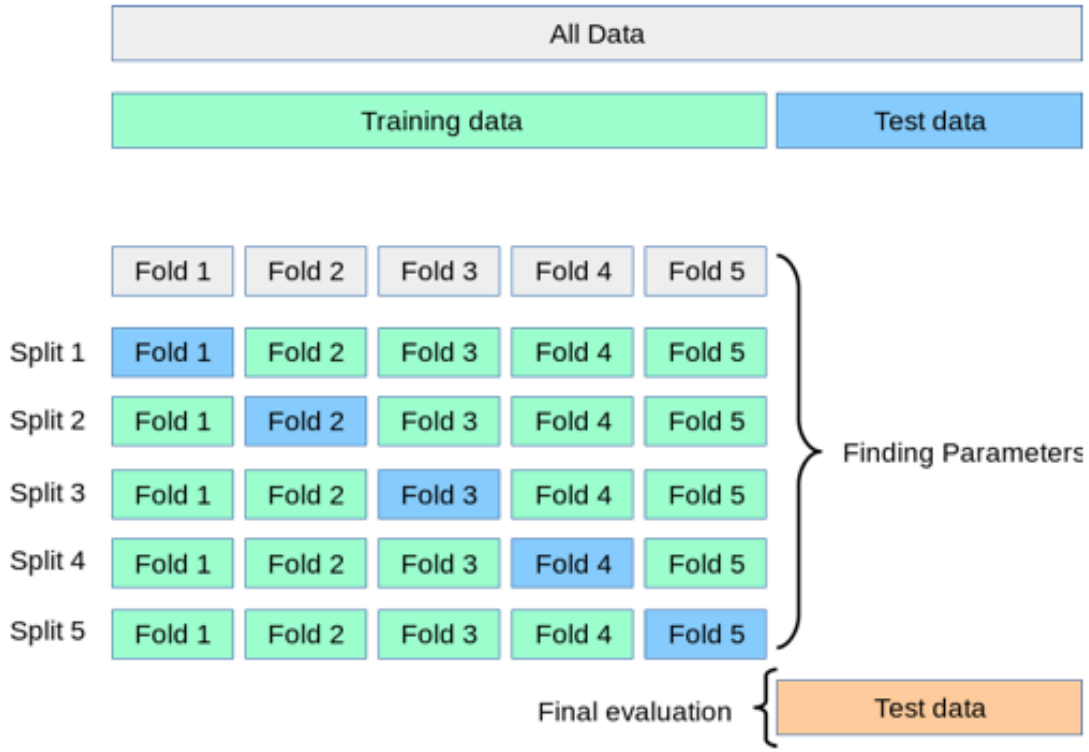


Select and Train a Model - Better Evaluation Using Cross-Validation

- Holdout part of the training set for validation
- After choosing best model (and model values)
  - Train the best model on the full training set
  - Evaluate final/best model on the test set -> generalization error

```
>>> pd.Series(tree_rmses).describe()
count       10.000000
mean     66868.027288
std       2060.966425
min      63649.536493
25%      65338.078316
50%      66801.953094
75%      68229.934454
max      70094.778246
dtype: float64
```

```
>>> pd.Series(forest_rmses).describe()
count       10.000000
mean     47019.561281
std       1033.957120
min      45458.112527
25%      46464.031184
50%      46967.596354
75%      47325.694987
max      49243.765795
dtype: float64
```



Fine-Tune the System - checklist

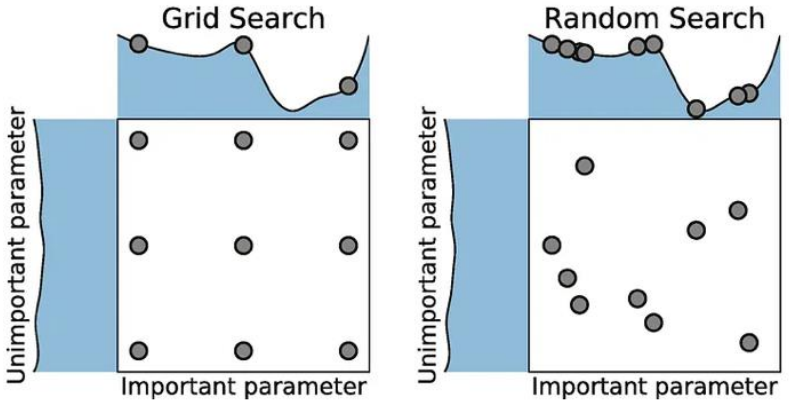
- You will want to use as much data as possible for this step, especially as you move toward the end of fine-tuning.
- As always, automate what you can.

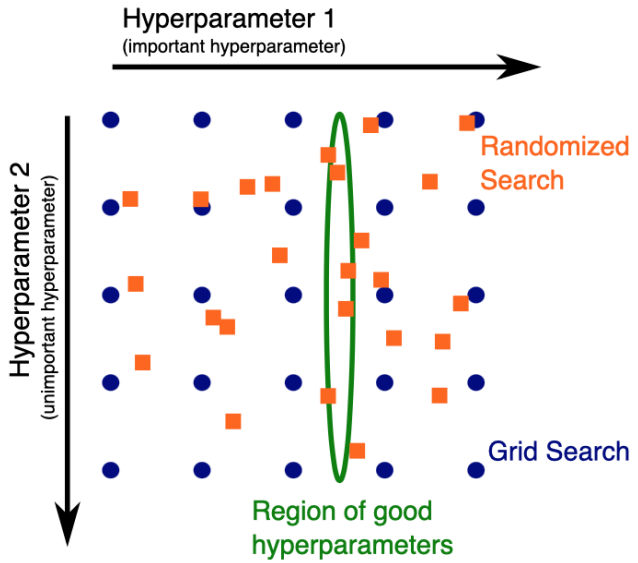
1. Fine-tune the hyperparameters using cross-validation:
   - Treat your data transformation choices as hyperparameters, especially when you are not sure about them (e.g., if you're not sure whether to replace missing values with zeros or with the median value, or to just drop the rows).
   - Unless there are very few hyperparameter values to explore, prefer random search over grid search. If training is very long, you may prefer a Bayesian optimization approach (e.g., using Gaussian process priors, as described by Jasper Snoek et al.1).
2. Try ensemble methods. Combining your best models will often produce better performance than running them individually.
3. Once you are confident about your final model, measure its performance on the test set to estimate the generalization error.

Fine-Tune Your Model

- Grid search
  - Use cross-validation to evaluate all the possible combinations of hyperparameter values
  - Fine when you are exploring relatively few combinations

- Randomized search
  - Evaluates a fixed number of combinations
  - Often preferable, especially when the hyperparameter search space is large





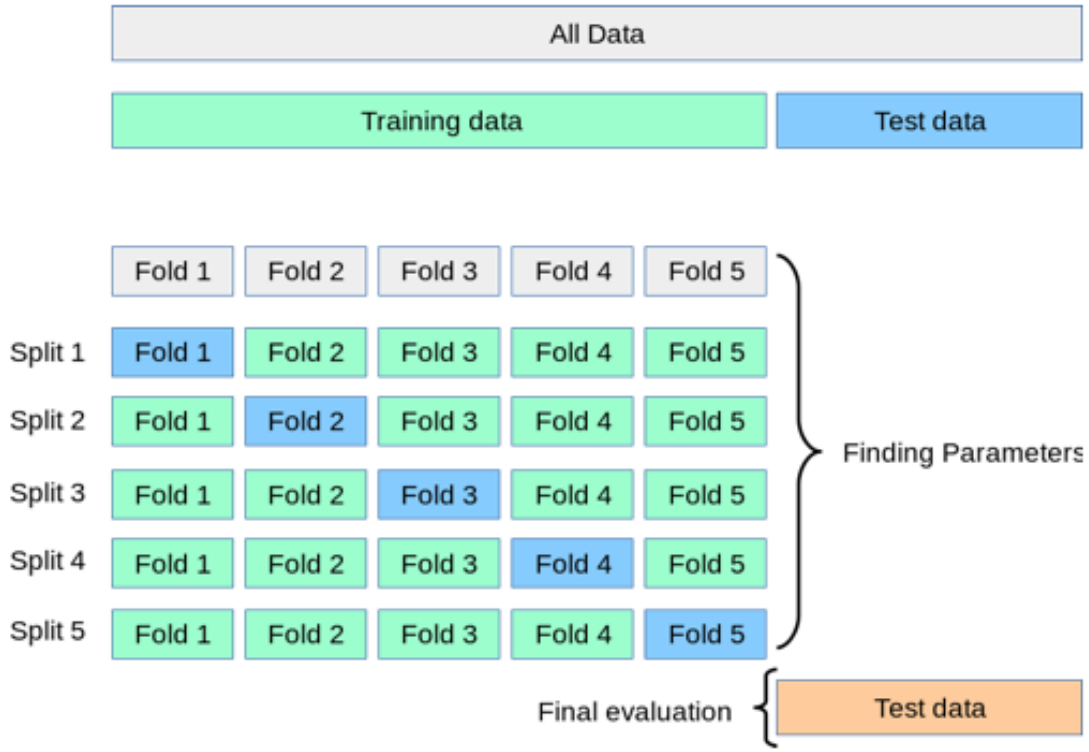
Analyzing the Best Models and Their Errors

- Inspect the best models
  - Can indicate the relative importance of each attribute for making accurate predictions
- Dropping some of the less useful features
  - Only one ocean_proximity category is really useful
- Look at the specific errors
  - What could fix the problem?
    - Adding extra features
    - Getting rid of uninformative ones
    - Cleaning up outliers
    - Etc.
- Ensure that your model not only works well on average!!!
  - Rural or urban districts, rich or poor, northern or southern, minority or not, etc.

```
>>> sorted(zip(feature_importances,
...            final_model["preprocessing"].get_feature_names_out()),
...            reverse=True)
...
[(0.18694559869103852, 'log__median_income'),
 (0.0748194905715524, 'cat__ocean_proximity_INLAND'),
 (0.06926417748515576, 'bedrooms__ratio'),
 (0.05446998753775219, 'rooms_per_house__ratio'),
 (0.05262301809680712, 'people_per_house__ratio'),
 (0.03819415873915732, 'geo__Cluster 0 similarity'),
 [...]
 (0.00015061247730531558, 'cat__ocean_proximity_NEAR BAY'),
 (7.301686597099842e-05, 'cat__ocean_proximity_ISLAND')]
```

Evaluate Your System on the Test Set

- You eventually have a system that performs sufficiently well
- Get the predictors and the labels from your test set
  - Transform the data and make predictions
  - Evaluate these predictions



Present Your Solution - checklist

1. Document what you have done.
2. Create a nice presentation:
   - Make sure you highlight the big picture first.
3. Explain why your solution achieves the business objective.
4. Don't forget to present interesting points you noticed along the way:
   - Describe what worked and what did not.
   - List your assumptions and your system's limitations.
5. Ensure your key findings are communicated through beautiful visualizations or easy-to-remember statements (e.g., "the median income is the number-one predictor of housing prices").

Launch! - checklist

1. Get your solution ready for production (plug into production data inputs, write unit tests, etc.).
2. Write monitoring code to check your system's live performance at regular intervals and trigger alerts when it drops:
   - Beware of slow degradation: models tend to "rot" as data evolves.
   - Measuring performance may require a human pipeline (e.g., via a crowdsourcing service).
   - Also monitor your inputs' quality (e.g., a malfunctioning sensor sending random values, or another team's output becoming stale). This is particularly important for online learning systems.
3. Retrain your models on a regular basis on fresh data (automate as much as possible).

Launch, Monitor, and Maintain Your System

- Get your solution ready for production
  - Save the best model you trained
  - Transfer the file to your production environment
  - Load it!

- Write monitoring code
  - Check your system's live performance at regular intervals and trigger alerts when it drops
  - If the model was trained with last year's data, it may not be adapted to today's data
  - How?
    - Inferred from downstream metrics (e.g. recommender systems)
    - Human analysis (e.g. image classification to detect product defects)
  - It is often much more work than building and training a model

Launch, Monitor, and Maintain Your System

- Update your datasets and retrain your model regularly - Automate as much as possible!
  - Collect fresh data regularly and label it (e.g., using human raters).
  - Write a script to train the model and fine-tune the hyperparameters automatically. This script could run automatically, for example every day or every week, depending on your needs.
  - Write another script that will evaluate both the new model and the previous model on the updated test set, and deploy the model to production if the performance has not decreased.

- Evaluate the model's input data quality
  - E.g., a malfunctioning sensor sending random values
  - Trigger an alert if more and more inputs are missing a feature, or the mean or standard deviation drifts too far from the training set, or a categorical feature starts containing new categories

- Keep backups of every model!

- Machine learning involves quite a lot of infrastructure – ML Operations (MLOps)
  - Once all the infrastructure is in place, going from idea to production will be much faster.

The Awful Truth

- Much of the work is in the data preparation step: building monitoring tools, setting up human evaluation pipelines, and automating regular model training.
- The machine learning algorithms are important, of course…
- …but it is probably preferable to be comfortable with the overall process and know three or four algorithms well rather than to spend all your time exploring advanced algorithms